(based on dataset popularity and ease of use) in ascertaining if a given dataset is a likely training dataset candidate.

Formally, we define the source dataset as $D_{\texttt{src}} = \{q_i, \texttt{src}_i\}_{i \in [N]}$ where (a) q_i is a problem instance within a task (e.g., name of a book for which a review has to be written), and (b) $\texttt{src}_i = \{\texttt{src}_i^j\}_{j \in [n_i]}$ is a set of n_i human-generated answers to the given prompt q_i (e.g., a set of n_i of book reviews for that particular book). In practice, we utilize existing datasets likely to be used during LLM training, and perform filtering and sub-sampling to obtain our $D_{\texttt{src}}$.

To create the model-generated dataset $D_{\texttt{gen}}$, for each sample q_i, we prompt the LLM we wish to evaluate, $\mathcal{M}$, m_i times to generate a set of responses, $\texttt{gen}_i = \{\texttt{gen}_i^j\}_{j \in [m_i]}$. Here, $\texttt{gen}_i^j \leftarrow \mathcal{M}_j(P_{\text{task}}(q_i), \texttt{kwargs})$ is the response obtained in the j-th call of $\mathcal{M}$, where (a) P_{task} denotes the task-specific formatting prompt that wraps the sample q_i, and (b) $\texttt{kwargs}$ denotes the generation keyword arguments (e.g., temperature) that specify the sampling strategy. Across both $D_{\texttt{src}}$ and $D_{\texttt{gen}}$, we select or generate a large enough number of responses per q_i to ensure variety.

4.2 Attribute Extraction

For a given task, we identify and compile a list of attributes that are of interest from the perspective of preserving diversity. This is a subjective task, but we focus on metrics which target understanding the content of LLM output (e.g., book review sentiment and topic; code time complexity and algorithms used), as opposed to more general metrics of output language quality such as saliency, fluency and coherence. More importantly, we need to ensure that extraction functions are efficient, accurate, and reproducible—we outline our tests to ensure these qualities in § 6 and the Appendices B and D. For example, care must be taken to use LLMs for attribute extraction, as they are known to be biased towards their own responses [24]. For a given attribute A, extraction function h_A takes a string $\texttt{tgt}$ to obtain the attribute value $h_A(\texttt{tgt})$. Note that $\texttt{tgt}$ can either be $\texttt{src}_i^j$ or $\texttt{gen}_i^j$. The extracted attribute can either be a continuous/categorical variable or of other more complicated types, depending on the nature of the attribute.

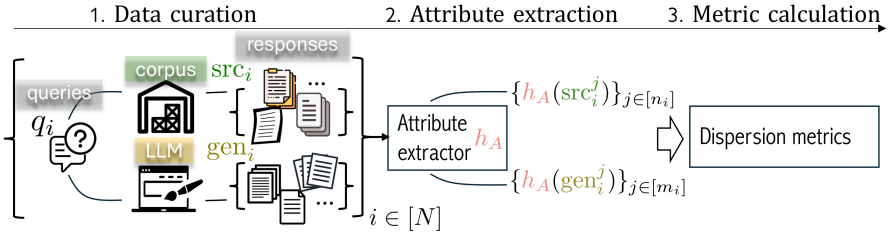

Figure 2: An overview of the procedure.

4.3 Metric Calculation

As the last step, we compute metrics on the extracted attributes. Given a set of responses, dispersion metrics aim to capture their breadth or coverage of these attributes. We describe those used in this paper below and in Fig. 3.

Dispersion Metrics. We introduce dispersion metrics suited to different data types.

A. Distribution of the mean. For ordinal or continuous attributes, we calculate the mean over the conditional distribution—that is, we calculate the metrics over each $\texttt{src}_i$ or $\texttt{gen}_i$ (e.g. reviews for a given book), and show the distribution of this mean over all q_i for a certain task (e.g. over all books). While the mean itself does not directly measure dispersion, the *distribution* of the mean values sheds light on dispersion: concentrated mean values indicates a smaller dispersion of the data. The advantage is that it not only describes dispersion, but also the qualitative tendency of the attribute (e.g. bias towards positive/negative sentiment), which cannot be captured otherwise.

B. Entropy and standard deviation. For categorical attributes, we measure dispersion using entropy over the conditional distribution. For continuous attributes, we use standard deviation over the conditional distribution to quantify the dispersion of values around the mean, providing the attribute's variability.

C. Mean pairwise similarity. For attributes that are not easily characterized as categorical or continuous, we adopt specific similarity metrics catered to the data type. We then calculate the pairwise similarity values for the conditional distribution—i.e. calculate the mean similarity value for all the pairs of elements within each $\texttt{src}_i/\texttt{gen}_i$, then show the distribution for all N samples. We use:

1. *Mean pairwise Jaccard index*: Given two sets of categorical variables A and B, the Jaccard index, $J(A, B) = {}^{|A \cap B|}/_{|A \cup B|}$, measures similarity between them. An example of such a set could be a set of several algorithms inferred from a piece of code. A higher mean Jaccard index indicates a higher overall similarity between the set, and consequently, lower dispersion.
2. *Mean pairwise cosine similarity*: Given two multi-dimensional embeddings e_1 and e_2 obtained via a sentence embedder [25], we calculate their similarity via cosine similarity, i.e., $S_C(e_1, e_2) = {}^{\langle e_1, e_2 \rangle}/_{\|e_1\| \|e_2\|}$. A higher mean cosine similarity indicates a higher similarity and lower dispersion.
3. *Mean pairwise fingerprint similarity*: For tasks related to coding or computer programs, similarity is based on the overlap of selected hash values (or fingerprints) generated by Winnowing [26]. We adopt an existing open-source tool COPYDETECT [27], which takes in a set of programs and returns the pairwise similarity scores for all programs in the set. We then calculate the mean value of these pairwise similarity scores as an indicator of the similarity for the set of programs. A higher mean fingerprint similarity indicateshigher structural and syntactical similarity of the code.

In addition to the dispersion metrics, we consider one other approach—visualizing the top modes of unconditional distributions for certain attributes, e.g., topics. This helps identify areas of emphasis in `src` and `gen` distributions, as well as the tendency of change across distributions.

5 Mitigating Generative Monoculture

To attempt to mitigate generative monoculture, we test four methods known to increase LLM output diversity: increasing the *temperature* T, top-p parameter, setting a temperature decay, and changing prompts. More details are in Appendices A and C.6.

Temperature T. This determines the dispersion of the probability distribution over the next token: increasing the temperature leads to a more flat probability distribution and increases the likelihood of sampling from less probable tokens, resulting in more diverse generations.

Top-p. This controls the randomness of the generations by limiting the range of tokens considered. Specifically, it considers the smallest subset (consisting of the top probability tokens) whose cumulative probability exceeds the threshold p. A smaller p encourages the model to sample from a more focused set of likely tokens.

Decaying Temperature. We choose the starting temperature $T = 10.0$ and follow a linear schedule for temperature decay, over the course of 50 time-steps (i.e., from the 1-st output token to the 50-th output token), with an ending temperature $T = 1.2$. The method is inspired by Carlini *et al.* [28].

Prompts. Tuning the specific content and framing of the prompt can steer the model's output more effectively [29, 30] and significantly impact the diversity of the generated text. We use "role-playing" or impersonation [31], which instructs the model to produce the output in the persona of a specific person, and expect it to induce more personalized and varied responses.

6 Experimental Setup

In this section, we describe our experimental setup for measuring and mitigating generative monoculture for two tasks, namely, generating book reviews and code solutions. We provide details for datasets, LLMs used, and most notably, the data attributes and metrics considered. We open source our code at `https://github.com/GeMoLLM/GeMO`.

6.1 Generating Book Reviews

Data Curation: For $D_{\texttt{src}}$, we use the Goodreads dataset [12], which contains multiple books with several reviews each. We perform filtering and sampling to ensure reliable attribute extraction

(see Appendix B.1), and craft a final dataset of $N = 742$ books with English titles, and $\forall i, n_i = 10$ reviews per book such that the review length is between 300 and 700 words.

To obtain $D_{\texttt{gen}}$, we used the following LLMs: (a) `Llama-2-13b` [32] (henceforth referred to as `Llama-2`), (b) `Llama-2-13b-chat` [32] (henceforth referred to as `Llama-2-chat`), (c) `Vicuna-13b-v1.5` [33] (henceforth referred to as `Vicuna-13b`), (d) `GPT-3.5-turbo-instruct (0914)` [17] (henceforth referred to as `GPT-3.5`), and (e) `GPT-4-turbo (0125)` [34, 13] (henceforth referred to as `GPT-4`). We performed nucleus sampling [35] with various sampling parameters: (a) temperature $T \in \{0.5, 0.8, 1.0, 1.2, 1.5\}$, and (b) top-$p \in \{0.90, 0.95, 0.98, 1.00\}$. We also experimented with two candidates for P_{task}: prompt (1) "`Write a personalized review of the book titled {title}:`", and prompt (2) "`Write a book review for the book titled {title} as if you are {person}:`". Prompt (2) was chosen as LLMs are known to generate more diverse responses when instantiated with a persona [31]. We list the names of the 10 persons we considered in Appendix B.2. For each combination of LLM, sampling parameter, and prompt, we independently sampled from the LLM 10 times to generate responses. We filtered out low-quality (generated) reviews by examining their perplexity (see Appendix C.1). This is to ensure that the data used for analysis represents well-formed and coherent text, thereby improving the reliability of our findings. Thus, $\forall i, m_i \leq 10$.

	Attribute	Data type	Level	Metric
Book review	sentiment	categorical	C	mean, entropy
	topic	categorical	C	entropy
			U	distribution visualization
	wording	categorical	U	count, entropy
Coding	correctness	categorical	C	mean
	efficiency (complexity)	categorical	C	entropy
			U	distribution visualization
	efficiency (runtime)	continuous	C	mean, standard deviation
	fingerprint	hash values	C	mean pairwise fingerprint similarity
	code summary (text)	embedding	C	mean pairwise cosine similarity
	code summary (categorical)	categorical	C	mean pairwise Jaccard index

Figure 3: A summary of the scenarios, the attributes we consider, their data types, and the corresponding analysis levels as well as metrics. C and U stand for conditional and unconditional distributions.

Attribute Extraction: We want attributes that capture both the semantics and syntax of book reviews. These attributes are representative of the key thematic and linguistic elements in book reviews. Most importantly, while these attributes are not exhaustive, their extraction is reliable and efficient.

1. *Sentiment* indicates whether a review is positive (praising the book) or negative (criticizing the book). We employ a fine-tuned sentiment classifier [36] as the attribute extractor which accepts a text and returns a prediction in $\{0, 1\}$. This model has been downloaded 5.4 million times in May 2024, and reaches an accuracy of 91.3 % on the `dev` set of SST-2 [37].
2. *Topic* refers to the themes discussed in a review [38, 39]. We leverage `BERTopic` [40] pre-trained on Wikipedia [41] which assigns one topic to each review out of a total of ∼2,000 topics.
3. *Word choice* captures the lexical diversity in a review. To quantify this, we produce a frequency table of the unique words (see Appendix B.3), and immediately have the number of unique words.

Metric Calculation: For sentiment, we calculate mean and entropy for the conditional distribution. For topic, we calculate entropy for the conditional distribution as well as visualize the unconditional distribution of topics across all reviews, focusing on the top 10 classes. Finally, for word choice, we calculate count and entropy of the unconditional distribution.

6.2 Generating Code Solutions

Data Curation: For $D_{\texttt{src}}$, we chose the CodeContests dataset [14], a competitive programming problem dataset where each problem comes with multiple correct and incorrect solutions. We limited the scope to a subset ($N = 100$) of level-A problems (easy problems) on Codeforces [42], and the language of the solutions to `python3`. More details in Appendix D.1. For each problem in the subset, we randomly sampled $\forall i, n_i = 20$ correct solutions from all of the n_i^{correct} solutions for that problem.

To obtain $D_{\texttt{gen}}$, we use the following LLMs: (a) `GPT-4`, and (b) `Claude-3-Sonnet` [43]. We did not use open-source LLMs, as these were not able to generate correct solutions for the problems we chose. More details are in Appendix E.4. We performed nucleus sampling [35] with various